人工智能中的编程第三次作业实验报告

CUDA 模块前向与反向传播实例测试

实验目标与环境

本实验实现全连接、普通二维卷积、2×2 最大池化、SoftMax 和交叉熵损失的核心计算，并用独立的 CPU 计算结果检查 GPU 输出。测试环境为 NVIDIA GeForce RTX 5060 Laptop GPU、CUDA Toolkit 13.0、MSVC 14.44；使用单精度浮点数。

核心实现

全连接层

输入 X 为 N×Cin，权重 W 为 Cout×Cin。前向计算 $Y=XW^T+b$；反向通过行主序 GEMM 包装调用 cublasSgemm，得到 $dW=dY^TX$、$dX=dYW$，并对 batch 维求和得到 db。权重用 cuRAND 初始化。

卷积层

采用 3×3、stride 1、zero padding 1。对每个样本，im2col 将输入展开为 (Cin×9)×(H×W)，再以 GEMM 计算 Y=Wcol，并加偏置。反向计算 $dW=dYcol^T$、$dcol=W^TdY$，再由 col2im 累加重叠窗口得到 dX。dW 与 db 在所有样本上累加。

最大池化层

每个 GPU 线程处理一个 2×2 窗口，输出最大值并用 0 至 3 的整数记录位置。反向传播把该窗口的输出梯度写回最大值所在的输入位置，其余三个位置为零。相等时选择按扫描顺序遇到的第一个最大值。

SoftMax 层

每个线程处理一个样本：先求该行最大值，减去最大值后计算指数与行和，再归一化得到各类别概率。另计算概率最大的预测类别。

交叉熵损失

各样本线程计算真实类别概率的负对数，用 atomicAdd 累加 batch 平均损失。计算前把概率下限设为 10^{-12}，避免 log(0)。与 SoftMax 合并后的梯度按 (p−one_hot)/N 逐行计算。

正确性测试与结果

正确性测试集中在 cuda_test.cu：用独立的 CPU 公式计算参考值，并将 GPU 结果逐元素比较。小规模测试的浮点绝对误差阈值为 10^{-4}；池化 mask 和预测类别要求完全相同。测试覆盖全连接、卷积普通及 1×1 边界、池化、SoftMax 和交叉熵。

模块	测试规模	检查项	结果
全连接	N=2，Cin=3，Cout=2	前向、dX、dW、db	全部通过
卷积	N=2，Cin=2，Cout=3，H=3，W=4	前向、dX、dW、db	全部通过
卷积边界	N=2，Cin=1，Cout=2，H=W=1	零填充边界及三种梯度	全部通过
最大池化	N=2，C=2，H=4，W=6	前向、反向、mask	全部通过
SoftMax	N=3，C=5；含较大正负 logits	概率及预测类别	全部通过
交叉熵	N=3，C=5	平均损失及合并梯度	全部通过

大规模正确性与运行时间对比

独立程序 cuda_benchmark.cu 在 RTX 5060 Laptop GPU 上测试。每项先预热 1 次，再重复 5 次取平均。CPU 采用启用 /O2 优化的朴素单线程循环；GPU 每次计算后同步。输入事先置于各自内存，计时不含初始主机与设备数据传输；GPU 计时包含模块内部的临时显存分配与释放，交叉熵还包含标量损失拷回主机。全连接、卷积、池化计入前向与反向；SoftMax 与交叉熵计入概率、损失和合并反向。

模块	输入规模	CPU 平均 ms	GPU 平均 ms	加速比
全连接	N=512，Cin=512，Cout=256	191.691	0.19898	963.4×
卷积	N=8，Cin=16，Cout=32，H=W=32	286.139	3.09046	92.6×
最大池化	N=64，C=64，H=W=64	60.6118	2.37122	25.6×
SoftMax 与交叉熵	N=32768，C=100	31.3798	0.70810	44.3×

大规模数据也逐元素与 CPU 对照：全连接的前向、dX、dW、db；卷积的前向、dX、dW、db；池化的前向与反向；SoftMax 概率、预测标签、交叉熵损失及合并梯度，全部通过。浮点比较采用 $|GPU-CPU| \le 0.002 + 0.001\times|CPU|$；预测标签完全相同。交叉熵损失的 CPU 和 GPU 结果均为 4.88166。

结论

五个模块的小规模正确性测试均通过；独立的大规模性能测试也通过 CPU 对照。较大的输入下，GPU 相对本报告所用的朴素单线程 CPU 参考实现更快。这些加速比仅对应本机、所选输入和实现，不能代表与优化 CPU 数值库的比较；测试仍未穷尽所有数值边界。